

Cốt truyện: "Những Mảnh Pha Lê Cuối Cùng"

(The Last Crystal Shards)

Bối cảnh thế giới

Vương quốc **Aeloria** từng được bảo vệ bởi **Tinh Thể Ánh Sáng** — một viên đá cổ đại nằm ở trung tâm thế giới, giữ cân bằng giữa ánh sáng và bóng tối. Hàng nghìn năm, con người, muông thú và các sinh linh sống hòa thuận dưới ánh sáng của nó.

Cho đến một ngày, **Kẻ Bóng Tối** — **Malachar** — phá vỡ Tinh Thể, tung các mảnh vỡ ra khắp bốn vùng đất: **Rừng Cổ Thụ**, **Thành Phố Bị Bỏ Hoang**, **Hang Động Dưới Lòng Đất**, và **Tháp Trời Vĩnh Cửu**. Không có Tinh Thể, bóng tối lan rộng, quái vật xuất hiện, và thế giới dần sụp đổ.

Nhân vật chính: Kael

Một cậu thanh niên bình thường sống ở ngôi làng nhỏ ven rừng. Làng bị quái vật tấn công và phá hủy trong đêm Tinh Thể vỡ. Kael là người duy nhất sống sót — và duy nhất có thể cảm nhận được ánh sáng yếu ớt còn sót lại trong các mảnh Tinh Thể.

"Nếu không ai đứng lên, thì ta sẽ là người đó."

Cấu trúc cốt truyện theo màn chơi

Màn	Vùng đất	Sự kiện chính
1–3	 Rừng Cổ Thụ	Kael rời làng, học cách chiến đấu và nhảy qua các cạm bẫy trong rừng. Thu thập mảnh Tinh Thể đầu tiên. Gặp tinh linh rừng Lyra — người trở thành đồng hành.
4–6	 Thành Phố Bị Bỏ Hoang	Thành phố hoang tàn, đầy cạm bẫy cơ học và lính bóng tối tuần tra. Kael khám phá ra Malachar từng là nhà vua — kẻ bị chính Tinh Thể từ chối vì lòng tham.
7–9	 Hang Động Dưới Lòng Đất	Hành trình vào hang tối, đầy hố sâu và quái vật. Lyra bị bắt. Kael phải một mình vượt qua thử thách tồi tệ nhất — và học được rằng mỗi lần ngã là một lần mạnh hơn.

Màn	Vùng đất	Sự kiện chính
10–12	 Tháp Trời Vĩnh Cửu	Leo lên đỉnh tháp nơi Malachar đang chờ. Trận chiến cuối cùng. Kael phải chọn: dùng Tinh Thể để phục hồi thế giới, hay dùng sức mạnh đó để trả thù?

Kết thúc (có hai hướng)

Kết thúc Ánh Sáng  : Kael ghép lại Tinh Thể, Malachar bị thanh lọc và trở về là con người bình thường. Thế giới hồi sinh. Kael từ chối trở thành vua — anh trở về xây lại làng cũ.

Kết thúc Bóng Tối  : Kael bị tha hóa bởi sức mạnh Tinh Thể trong hành trình. Anh đánh bại Malachar nhưng giữ lại Tinh Thể cho riêng mình — và trở thành kẻ thống trị mới mà anh từng căm ghét.

Gameplay ↔ Cốt truyện liên kết

Cơ chế game	Ý nghĩa cốt truyện
Nhảy đôi	Sức mạnh từ mảnh Tinh Thể đầu tiên thu thập được
Coin/Pickup	Tàn tích của người dân Aeloria để lại
Chết & hồi sinh	Lyra dùng phép thuật hồi sinh Kael — nhưng có giới hạn
Tilemap đa dạng	Bốn vùng đất đại diện cho bốn phần của Tinh Thể vỡ

2. Bổ sung các hiệu ứng âm thanh phù hợp: nhặt vật phẩm, chiến thắng,...
- Tạo “trung tâm âm thanh” trong GameManager để phát hiệu ứng bằng PlayOneShot.
 - Gọi âm thanh nhặt đồ trong pickup.
 - Gọi âm thanh nhảy trong PlayerController.
 - Gọi âm thanh chiến thắng trong ExitTrigger và chặn phát lặp nhiều lần.

// GameManager.cs (thêm biến + hàm phát SFX)

```
[Header("SFX")]
```

```
[SerializeField] private AudioSource sfxSource;
```

```
[SerializeField] private AudioClip pickupSfx;
```

```
[SerializeField] private AudioClip jumpSfx;
```

```
[SerializeField] private AudioClip winSfx;
```

```
[SerializeField, Range(0f, 1f)] private float pickupVolume = 0.9f;
```

```
[SerializeField, Range(0f, 1f)] private float jumpVolume = 0.85f;
```

```
[SerializeField, Range(0f, 1f)] private float winVolume = 1f;
```

```
private void Start()
```

```
{
```

```
    EnsureAudioSource();
```

```
    // ...
```

```
}
```

```
public void PlayPickupSfx() => PlaySfx(pickupSfx, pickupVolume);
```

```
public void PlayJumpSfx()  => PlaySfx(jumpSfx, jumpVolume);
```

```
public void PlayWinSfx()   => PlaySfx(winSfx, winVolume);
```

```
private void EnsureAudioSource()
```

```
{
```

```
    if (sfxSource == null) sfxSource = GetComponent<AudioSource>() ??  
    gameObject.AddComponent<AudioSource>();
```

```
    sfxSource.playOnAwake = false;
```

```
}
```

```
private void PlaySfx(AudioClip clip, float volume)
{
    if (clip == null || sfxSource == null) return;
    sfxSource.PlayOneShot(clip, volume);
}
```

// pickup.cs (khi nhặt coin/gem)

```
GameManager.instance.IncrementCoinCount();
GameManager.instance.PlayPickupSfx();
```

// PlayerController.cs (trong Jump)

```
private void Jump(float jumpForce)
{
    rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0);
    rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    playeranim.SetTrigger("jump");

    if (GameManager.instance != null)
        GameManager.instance.PlayJumpSfx();
}
```

// ExitTrigger.cs (phát win 1 lần)

```
private bool hasTriggered = false;

private void OnTriggerEnter2D(Collider2D collision)
{

```

```

if (hasTriggered) return;
if (collision.gameObject.tag == "Player")
{
    hasTriggered = true;
    if (GameManager.instance != null)
        GameManager.instance.PlayWinSfx();
    StartCoroutine("LevelExit");
}
}

```

Câu 3. Bổ sung cơ chế bẫy, kẻ thù

1. Cơ chế bẫy (HazardTrap.cs)

- Bẫy gây sát thương khi nhân vật chạm vào.
- Hỗ trợ cả va chạm dạng Trigger và Collision.
- Có biến damageCooldown để tránh trừ máu liên tục theo từng frame.
- Có tùy chọn instantKill cho các bẫy đặc biệt (hố, lava, spike nguy hiểm cao).

2. Cơ chế kẻ thù tuần tra (EnemyPatrol.cs)

- Kẻ thù di chuyển qua lại giữa hai mốc leftPoint và rightPoint.
- Tự đổi hướng khi đến điểm giới hạn.
- Tự lật sprite theo hướng di chuyển để chuyển động tự nhiên hơn.

3. Cơ chế tương tác người chơi - kẻ thù (EnemyContact.cs)

- Khi chạm ngang vào enemy: người chơi bị mất máu.
- Khi nhảy đập từ trên đầu enemy: enemy bị tiêu diệt, người chơi được bật nảy lên.
- Có damageCooldown để chống trừ máu quá nhanh khi đứng sát enemy.
- Hỗ trợ hiệu ứng chết (deathEffect) khi enemy bị tiêu diệt.

4. Điều chỉnh hệ thống máu (HealthManager.cs)

- Đã sửa hàm HurtPlayer() để khi máu giảm về 0 thì nhân vật chết ngay, không cần nhận thêm một đòn nữa.
- Hiệu ứng nhận sát thương vẫn được giữ nguyên.

Đoạn code tiêu biểu (trích)

```
// HazardTrap.cs
```

```
if (instantKill)
```

```
{
```

```
    GameManager.instance.Death();
```

```
}
```

```
else
```

```
{
```

```
    HealthManager.instance.HurtPlayer();
```

```
}
```

```
// EnemyContact.cs (đạp đầu)
```

```
if (canBeStomped && IsStompHit(collision, playerRb))
```

```
{
```

```
    StompKill(playerRb); // enemy chết, player nảy lên
```

```
    return;
```

```
}
```

```
DamagePlayer();
```

```
// EnemyPatrol.cs (tuần tra trái-phải)
```

```
Transform target = movingRight ? rightPoint : leftPoint;
```

```
float direction = Mathf.Sign(target.position.x - transform.position.x);
```

```
rb.linearVelocity = new Vector2(direction * moveSpeed, rb.linearVelocity.y);
```

Kết quả

Sau khi bổ sung, game có thêm chương ngại động và cơ chế nguy hiểm rõ ràng:

- Người chơi phải né bẫy, căn thời điểm di chuyển.
- Tương tác với enemy đa dạng hơn (né, chịu damage, hoặc đập đầu tiêu diệt).
- Gameplay trở nên hấp dẫn và đúng đặc trưng thể loại platformer hơn.

4. Tùy chỉnh UI, xây dựng thêm menu About, khi người dùng click vào menu About, hệ thống hiển thị thông tin bao gồm: Tên Game, Mô tả ngắn, Danh sách thành viên nhóm